

Lecture 02 - CSS Fundamentals & the Box Model

Dr Andy Guest

Lecture Overview

- Why CSS exists
- Separation of content and presentation
- Three ways to apply CSS
- CSS syntax and selectors
- Typography and colour
- The box model
- Margins, padding, and borders
- Specificity and the cascade
- CSS validation
- Lab: styling the bio page

Separating Content from Presentation

The Problem CSS Solves

Before CSS, presentation was mixed directly into HTML.

The old way (deprecated):

```
<font size="7" color="red">  
  Big red heading  
</font>  
  
<p align="center">  
  Centred paragraph  
</p>  
  
<table bgcolor="yellow">
```

Every page had to be styled individually.
Change the colour? Edit every file.

The problems:

- Presentation tangled with content
- Repetition across every page
- Impossible to maintain at scale
- Accessibility and standards failures

The solution:

Separate *what* content is (HTML) from *how* it looks (CSS).

One stylesheet can control an entire site.

Separation of Concerns

A core principle of web development — and software engineering generally.

Layer	Technology	Responsibility
Structure	HTML	What things <i>are</i>
Presentation	CSS	How things <i>look</i>
Behaviour	JavaScript	How things <i>act</i>

In practice:

Changing the visual design of a site should never require touching the HTML.

Changing the content should never require touching the CSS.

This is why external stylesheets exist.

Each layer does one job.

Each can be changed independently.

Three Methods

Method 1 — Inline Styles

Applied directly to a single HTML element using the `style` attribute.

```
<h1 style="color: navy; font-size: 2em;">Welcome</h1>  
<p style="background-color: lightyellow; padding: 10px;">Hello world.</p>
```

When it applies:

Only to that specific element on that specific page.

Use case:

Rarely justified — mostly useful for quick testing or overriding styles in exceptional cases.

Problems:

- Mixes presentation into HTML
- Cannot be reused
- Hardest to maintain
- Overrides everything else (highest specificity)

Avoid in production code.

Method 2 — Internal Stylesheet

Defined in the `<head>` of a single HTML document using `<style>` tags.

```
<head>
  <style>
    body {
      background-color: #f5f5f5;
      font-family: Arial, sans-serif;
    }
    h1 {
      color: navy;
      text-align: center;
    }
  </style>
</head>
```

Scope: The entire page it is written in.

Use case: Single-page sites; quick prototyping; styles that genuinely apply to one page only.

Problems:

- Does not apply across multiple pages
- Still mixes HTML and CSS in one file
- Harder to maintain as sites grow

Method 3 — External Stylesheet

CSS written in a separate `.css` file, linked from every HTML page.

```
<head>
  <link rel="stylesheet" href="style.css">
</head>
```

```
/* style.css */
body {
  background-color: #f5f5f5;
  font-family: Arial, sans-serif;
}

h1 {
  color: navy;
  text-align: center;
}
```

Scope: Every page that links to it.

Use case: All real websites. This is the

Advantages:

- One file controls the whole site

Which Method to Use?

Method	Scope	Use When
Inline	One element	Almost never
Internal	One page	Prototyping only
External	Whole site	Always, in production

The rule of thumb:

For any site with more than one page, use an external stylesheet.

For your portfolio assessment, external stylesheets are expected.

Inline styles in submitted work will be penalised under Web Standards.

Rules, Selectors, and Properties

CSS Rule Syntax

A CSS **rule** consists of a selector and a declaration block.

```
selector {  
  property: value;  
  property: value;  
}
```

Example:

```
h1 {  
  color: navy;  
  font-size: 2em;  
  text-align: center;  
}  
  
p {  
  font-family: Arial, sans-serif;  
  line-height: 1.6;  
}
```

Anatomy:

- **Selector** — which elements to target
- **Property** — what to change (`color` , `font-size`)
- **Value** — what to set it to (`navy` , `2em`)
- **Declaration** — one property/value pair
- **Declaration block** — all declarations in {
}
- Declarations end with a semicolon ;

Selectors — Element, Class, and ID

The three fundamental selector types.

Element selector

Targets all elements of that type:

```
p {  
  color: #333;  
}  
  
h2 {  
  font-weight: bold;  
}
```

Applies to every `<p>` or `<h2>` on the page.

Class selector (`.`)

Targets elements with a matching `class` attribute:

```
.highlight {  
  background-color: yellow;  
}  
  
.intro {  
  font-size: 1.2em;  
}
```

```
<p class="intro">Welcome...</p>  
<p class="highlight">Important!</p>
```

Multiple elements can share a class.

Selectors — Element, Class, and ID (continued)

ID selector (#)

Targets a single element with a matching `id` attribute:

```
#main-heading {  
  color: darkred;  
  border-bottom: 2px solid darkred;  
}
```

```
<h1 id="main-heading">Welcome</h1>
```

An `id` must be **unique** on the page.
Use IDs sparingly — classes are more flexible.

When to use which:

Selector	Use for
Element	Default styles for all instances
Class	Reusable styles, multiple elements
ID	Unique page elements (e.g. <code>#header</code>)

Prefer classes over IDs for styling.
IDs are better suited to JavaScript anchors.

Controlling How Text Looks

Font Families

CSS organises fonts into five generic families.

```
/* Serif – formal, traditional */
p.serif {
  font-family: "Times New Roman", Times, serif;
}

/* Sans-serif – clean, modern */
p.sans {
  font-family: Arial, Helvetica, sans-serif;
}

/* Monospace – code, terminals */
p.mono {
  font-family: "Courier New", Courier, monospace;
}
```

Always provide fallbacks:

```
font-family: "Preferred Font", Fallback, generic;
```

If the preferred font isn't installed, the browser moves to the next.

The generic family (`serif`, `sans-serif`, `monospace`) is the final fallback.

For screens:

Sans-serif fonts are generally more legible at small sizes on screen.

Font Size, Weight, and Style

Size:

```
h1 { font-size: 2em; }      /* relative to parent */
h2 { font-size: 1.5rem; }   /* relative to root */
p  { font-size: 16px; }     /* absolute pixels */
p  { font-size: 100%; }     /* percentage */
```

Prefer `em` or `rem` over `px` for accessibility
— users can scale text in their browser.

Weight:

```
p.light { font-weight: 300; }
p.normal { font-weight: normal; } /* 400 */
p.bold { font-weight: bold; } /* 700 */
```

Style:

```
p.normal { font-style: normal; }
p.italic { font-style: italic; }
```

Text decoration:

```
a { text-decoration: none; }
del { text-decoration: line-through; }
.underline { text-decoration: underline; }
```

Text alignment:

```
h1 { text-align: center; }
p { text-align: left; }
p { text-align: justify; }
```

Colour

CSS supports multiple colour formats — all equivalent, choose what fits your workflow.

Named colours (140 built-in):

```
color: tomato;  
color: dodgerblue;  
color: mediumseagreen;
```

Hexadecimal (#RRGGBB):

```
color: #ff0000; /* red */  
color: #0000ff; /* blue */  
color: #3a3a3a; /* dark grey */
```

RGB:

```
color: rgb(255, 0, 0); /* red */  
color: rgb(58, 58, 58); /* dark grey */
```

RGBA (with transparency):

```
background-color: rgba(0, 0, 0, 0.5); /* 50% black */
```

HSL (hue, saturation, lightness):

```
color: hsl(220, 80%, 40%);
```

Which to use?

Hex is most common. RGBA is useful for overlays. HSL is easiest to reason about

when adjusting shades.

How Every Element Takes Up Space

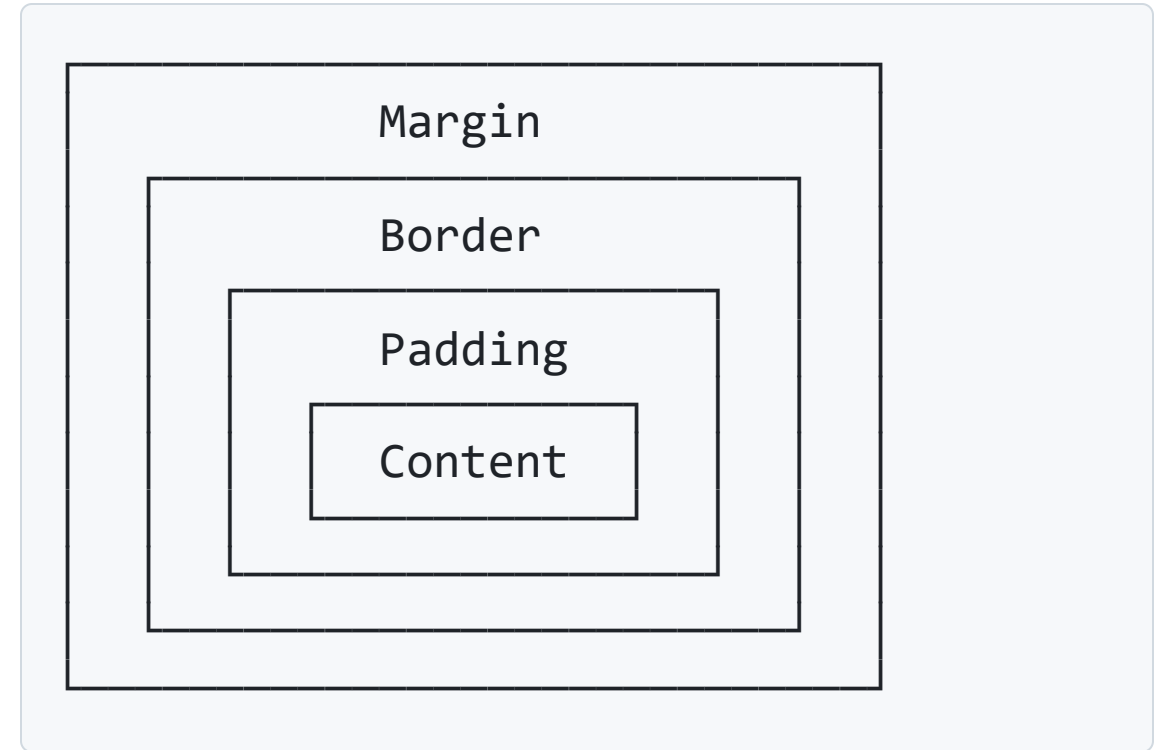
Everything is a Box

In CSS, every HTML element is rendered as a rectangular box.

The box has four layers, from inside out:

1. **Content** — text, images, child elements
2. **Padding** — space inside the border
3. **Border** — the element's visible edge
4. **Margin** — space outside the border

Understanding this model is fundamental to layout.



Padding

Space between the content and the border. Always inside the element.

Individual sides:

```
p {  
  padding-top: 20px;  
  padding-right: 40px;  
  padding-bottom: 20px;  
  padding-left: 40px;  
}
```

Shorthand (top, right, bottom, left — clockwise):

```
p { padding: 20px 40px 20px 40px; }  
  
/* top/bottom | left/right */  
p { padding: 20px 40px; }  
  
/* all sides equal */  
p { padding: 20px; }
```

Key points:

- Padding adds to the element's visible size
- It inherits the element's background colour
- Clicking within the padding area still triggers events

```
/* A card-style block */  
.card {  
  background-color: #f0f4ff;  
  padding: 24px 32px;  
  border-radius: 8px;  
}
```

Margin

Space outside the border — separates the element from its neighbours.

Individual sides:

```
p {  
  margin-top: 16px;  
  margin-right: 0;  
  margin-bottom: 16px;  
  margin-left: 0;  
}
```

Shorthand:

```
/* top/bottom | left/right */  
p { margin: 16px 0; }  
  
/* all sides */  
p { margin: 16px; }  
  
/* centre a block element horizontally */  
.container {  
  width: 800px;
```

Margin collapse:

Adjacent vertical margins between elements *collapse* to the larger value — they do not add together.

```
p { margin-bottom: 20px; }  
h2 { margin-top: 30px; }  
/* gap between them = 30px, not 50px */
```

This is intentional behaviour — worth understanding before it surprises you.

Borders

Three required properties:

```
p {  
  border-style: solid;  
  border-width: 2px;  
  border-color: #333;  
}
```

Shorthand:

```
p { border: 2px solid #333; }
```

Border styles:

```
border-style: solid;  
border-style: dashed;  
border-style: dotted;  
border-style: double;  
border-style: none;
```

Individual sides:

```
.box {  
  border-top: 3px solid navy;  
  border-bottom: 1px dashed grey;  
  /* no left or right border */  
}
```

Rounded corners:

```
.card {  
  border: 1px solid #ddd;  
  border-radius: 8px; /* all corners */  
  border-radius: 4px 16px; /* top-bottom | left-right */  
}
```

box-sizing

By default, `width` and `height` apply to the **content area only** — padding and border are added on top.

Default behaviour (`content-box`):

```
.box {  
  width: 200px;  
  padding: 20px;  
  border: 2px solid black;  
}  
/* Total rendered width: 200 + 40 + 4 = 244px */
```

This is counterintuitive and causes layout surprises.

Better behaviour (`border-box`):

```
*, *::before, *::after {  
  box-sizing: border-box;  
}  
  
.box {  
  width: 200px;  
  padding: 20px;  
  border: 2px solid black;  
}  
/* Total rendered width: exactly 200px */
```

`border-box` includes padding and border in the declared width.

Apply it globally at the top of every

stylesheet

Which Rule Wins?

The Cascade

CSS stands for *Cascading* Style Sheets. The cascade determines which rule applies when multiple rules target the same element.

Rules are evaluated in this order:

1. **Origin** — browser defaults < author styles < inline styles
2. **Specificity** — how precisely the selector targets the element
3. **Source order** — later rules override earlier ones

Source order example:

```
p { color: blue; }  
p { color: red; }  
/* Paragraphs will be red – last rule wins */
```

Inheritance:

Some properties (e.g. `color`, `font-family`) are inherited by child elements.

Others (e.g. `border`, `padding`) are not.

Specificity

When multiple rules target the same element, the most *specific* selector wins.

Specificity is calculated as a score:

Selector type	Points
Inline style	1000
ID (<code>#id</code>)	100
Class (<code>.class</code>)	10
Element (<code>p</code> , <code>h1</code>)	1

Higher score = higher priority.

Examples:

```
p { color: blue; }           /* score: 1 */
.intro { color: green; }     /* score: 10 */
#hero { color: red; }       /* score: 100 */
```

```
<p class="intro" id="hero">
  Which colour wins?
</p>
<!-- Red – the ID selector wins -->
```

Practical advice:

Keep specificity low. Prefer classes over IDs for styling. Avoid `!important` — it makes debugging very painful.

CSS Validation

Like HTML, CSS should be validated against W3C standards.

W3C CSS Validator:

jigsaw.w3.org/css-validator

Validates by:

- Direct input
- URL (validates live site)
- File upload

Why validate?

- Catches typos in property names and values
- Identifies unsupported or incorrect syntax
- Required for the Web Standards criterion
- Validation logos should be displayed in your portfolio site

```
<!-- CSS validation logo -->
<a href="https://jigsaw.w3.org/css-validator/check/referer">
  
</a>
```

Key Takeaways

- CSS controls *presentation* — HTML controls *structure*. Keep them separate
- External stylesheets are the standard approach for any multi-page site
- Every CSS rule has a selector, properties, and values
- The three selector types — element, class, ID — differ in specificity and reuse
- Every element is a box: content → padding → border → margin
- `box-sizing: border-box` makes layout predictable — use it globally
- The cascade resolves conflicts: specificity > source order
- Validate your CSS at jigsaw.w3.org/css-validator

References

MDN Web Docs (2024) *CSS: Cascading Style Sheets*. Mozilla. Available at: <https://developer.mozilla.org/en-US/docs/Web/CSS> (Accessed: [date]).

Meyer, E. and Weyl, E. (2023) *CSS: The definitive guide*. 5th edn. Sebastopol: O'Reilly Media.

W3C (2023) *CSS Snapshot 2023*. World Wide Web Consortium. Available at: <https://www.w3.org/TR/css-2023> (Accessed: [date]).

W3Schools (2024) *CSS tutorial*. Available at: <https://www.w3schools.com/css> (Accessed: [date]).

Duckett, J. (2011) *HTML & CSS: Design and build websites*. Indianapolis: John Wiley & Sons.

Robbins, J.N. (2018) *Learning web design: A beginner's guide to HTML, CSS, JavaScript and web graphics*. 5th edn. Sebastopol: O'Reilly Media.